

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

Introdução ao controle de versão

Aula 3

Código da aula: [DADOS]ANO1C1B1S4A3

Exposição



Objetivo da aula

- Apresentar conceitos elementares do controle de versão.



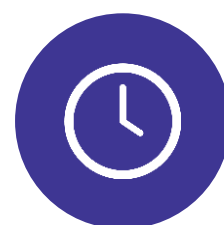
Competências da unidade (técnicas e socioemocionais)

- Usar ferramentas de desenvolvimento de software, como Git e GitHub;
- Trabalhar em equipe, compartilhando conhecimentos, contribuindo com ideias e colaborando para alcançar objetivos comuns.



Recurso didático

- Recurso audiovisual para a exibição de vídeos e imagens.



Duração da aula

50 minutos.

Conceitos importantes em controle de versão: repositórios

A base do Git está no conceito de **repositórios**, que nos permitem armazenar todos os arquivos importantes para o nosso programa e controlar a versão do que estamos desenvolvendo. Para isso, cada mudança também deve ser registrada. Durante cada registro de mudança, é necessário um processo de mesclagem dessas “novas” informações.

Imagine que você está desenvolvendo um novo website e tem várias pessoas na equipe trabalhando em diferentes partes do projeto. Para organizar isso, você cria um repositório no GitHub. Dentro desse repositório, você tem todos os arquivos relacionados ao seu site: código HTML, folhas de estilo CSS, scripts JavaScript e qualquer outra coisa.

Repositórios

Repositório local: é como ter uma cópia do álbum de fotos no seu próprio computador. Você pode abrir o álbum, adicionar novas fotos e até mesmo fazer algumas edições. Esse é o seu espaço de trabalho local.

Repositório remoto: agora, imagine que você queira compartilhar seu álbum com outras pessoas. Você cria uma versão on-line do seu álbum (um repositório remoto) em um site como o FotoHub. Outras pessoas podem ver suas fotos, fazer cópias locais e até mesmo adicionar suas próprias fotos.

Resumindo repositórios

O que eles permitem?

Histórico de versões

Sempre que alguém adiciona ou altera uma foto, o álbum registra quem fez isso e quando. Isso é como um histórico para acompanhar todas as mudanças.

Ramos (*branches*)

Às vezes, você quer experimentar novas ideias sem mexer nas fotos principais. Então, você cria cópias temporárias do álbum (ramos) para testar diferentes coisas.

Colaboração

Quando várias pessoas trabalham juntas em um projeto, todas elas têm acesso ao mesmo álbum on-line. Isso facilita a colaboração, pois todos estão olhando para a mesma coleção de fotos.

Sincronização

Se alguém faz uma alteração on-line, como adicionar uma foto nova, todos podem atualizar seus álbuns locais para ver a última versão.

Conceitos de controle de versão: *commit* (registro de versão)

O termo *commit*, em desenvolvimento de software, refere-se a uma ação específica no sistema de controle de versão, como Git.

Um *commit* é uma confirmação ou registro de uma mudança feita no código-fonte ou em outros arquivos do projeto.

Pode ser útil pensar em um *commit* como uma captura de tela do estado atual do projeto em um determinado momento.

O que compõe um *commit*?

Registro de mudanças: um *commit* seria como uma pessoa fazendo uma entrada específica no caderno. Digamos que alguém anotou: "Adiciona funcionalidade de login".

Mensagem de *commit*: essa anotação é acompanhada de uma mensagem explicativa, como se fosse uma nota na margem do caderno, que diz: "Agora podemos fazer login no sistema".

Histórico do projeto: cada vez que alguém faz uma entrada no caderno, isso é como um novo *commit*. As páginas do caderno representam diferentes momentos no tempo, mostrando como as informações evoluíram ao longo do projeto.

Exposição

O que o *commit* nos possibilita?

Reversão e investigação: se alguém perceber que a adição do login causou um problema, é possível folhear o caderno para encontrar a entrada específica que adicionou essa funcionalidade. Se decidirem que era melhor antes, podem remover ou "reverter" essa entrada.

Colaboração: cada pessoa pode adicionar suas próprias entradas no caderno (fazer *commits*) em diferentes seções, mas todos compartilham o mesmo caderno, o que facilita a colaboração.

Conceitos de controle de versão: *branches* (ramificações)

Em resumo, *branches* permitem que os desenvolvedores isolem seu trabalho, experimentem novas ideias e colaborem eficientemente, mantendo a integridade do projeto principal.

Cada *branch* é como uma seção no caderno de anotações, proporcionando uma abordagem organizada para o desenvolvimento de software.

O que uma *branch* nos permite?

Ramos como seções no caderno: imagine que o caderno de anotações é o projeto principal e cada *branch* é como uma seção separada do caderno. Cada seção pode representar uma ideia, uma nova funcionalidade ou uma correção de bugs.

Isolamento de trabalho: quando você cria uma nova seção (*branch*) no caderno, pode trabalhar nela sem interferir no conteúdo das outras seções. Isso significa que, se você quiser experimentar algo novo ou corrigir um problema, pode fazer isso em sua própria seção sem afetar o resto do caderno.

O que uma *branch* nos permite?

Mudanças independentes: suponha que você e um colega estejam trabalhando em diferentes seções do caderno. Vocês podem fazer alterações independentes nas suas próprias seções (*branches*) sem atrapalhar o trabalho um do outro.

Testes separados: cada seção (*branch*) pode ser vista como um teste ou experimento separado. Se algo der errado ou se a ideia não funcionar, você pode descartar toda a seção sem afetar o conteúdo principal do caderno.

Histórico separado: cada seção (*branch*) mantém seu próprio histórico de alterações. Isso é útil para entender como uma ideia ou funcionalidade específica evoluiu ao longo do tempo.

O que uma *branch* nos permite?

Mesclagem (*merge*): quando você estiver satisfeito com o que fez em sua seção (*branch*) e quiser adicionar isso ao caderno principal, você pode "mesclar" sua seção de volta ao caderno. Isso significa incorporar suas alterações ao projeto principal.

Colaboração eficiente: se várias pessoas na equipe estão trabalhando em diferentes aspectos do projeto, cada uma pode ter sua própria seção (*branch*) no caderno. Isso facilita a colaboração, pois as mudanças podem ser integradas ao projeto principal de maneira organizada.

Vamos
fazer uma
atividade

Elaborem um documento de texto comentando todos os itens apresentados.



15 min



Em duplas ou trios

Com base nas aulas anteriores, discorra sobre:

- 1** A importância do controle de versão para o desenvolvimento acelerado de novas tecnologias.
- 2** A importância de criar *branches* em um projeto.
- 3** A importância de o software identificar os conflitos entre duas versões de um arquivo em meio ao desenvolvimento de programas complexos.



O que nós
**aprendemos
hoje?**

© Getty Images

Hoje desenvolvemos:

- 1** A compreensão dos repositórios como “armários” de projeto, onde documentos e rascunhos são guardados e organizados para acesso da equipe.
- 2** A analogia dos *branches* em repositórios, comparando-os a seções específicas de um armário para diferentes aspectos do projeto.
- 3** A noção de *commits* como registros de mudanças significativas, semelhantes a tirar uma foto do estado atual de uma seção e anotar o que foi alterado.



Saiba mais

Neste vídeo, exploraremos o controle de versão, essencial no desenvolvimento de software profissional. Abordaremos como ele auxilia equipes de desenvolvimento a gerenciar e rastrear mudanças no código-fonte.

O que é controle de versão?

O controle de versão refere-se ao gerenciamento do histórico de modificações em softwares, permitindo controle efetivo sobre o código-fonte.

Assista ao vídeo para uma visão geral:

CELSONO KITAMURA. O que é controle de versão. Disponível em: https://www.youtube.com/watch?v=dzAoqq_w8kY. Acesso em: 16 jan. 2024.

Referências da aula

AQUILOS, A.; FERREIRA, R. **Controlando versões com Git e GitHub**. São Paulo: Casa do Código, 2014.

CHACON, S.; STRAUB, B. **Pro Git**. [S.l.]: Apress, 2014. E-book. Disponível em: <https://git-scm.com/book/pt-br/v2>. Acesso em: 16 jan. 2024.

Identidade visual: imagens © Getty Images

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**